University of Waterloo
Faculty of Engineering
Department of Electrical & Computer Engineering

Low Power Hardware Cryptography

Group 2016.020

(names removed)

Carlos Moreno **Consultant**

June 1, 2015

This is an excellent report, scoring 49 / 50

Contents

1 High Level Description **2**
1.1 Motivation . . . 2
1.2 Project Objective . . . 2
1.3 Block Diagram . . . 2
1.3.1 Noise Detector and Amplifier . . . 4
1.3.2 Noise Whitener (Hashing Block Instance) . . . 4
1.3.3 Key Arbiter . . . 4
1.3.4 Key Refresher . . . 4
1.3.5 Hashing (SHA-256) . . . 4
1.3.6 Message Authentication (Poly1305-Salsa20) . . . 4
1.3.7 Symmetric Cryptography (Salsa20) . . . 4
1.3.8 Asymmetric Cryptography (Curve25519) . . . 5
1.3.9 NaCL-like API . . . 5
1.3.10 SPI Slave Interface . . . 5

2 Project Specifications **5**

3 Risk Assessment **8**

1 High Level Description

1.1 Motivation

Embedded devices are becoming increasingly common in society. Historically, security and cryptography in these devices has been either ineffective or entirely absent. The real constraints of power usage and responsiveness have outweighed the hypothetical constraint of security. However, as these devices gain Internet connectivity they gain exposure to an ever more hostile world.

HP has recently found that 70% of Internet-of-Things (IoT) devices connect to network services without authentication [1]. A consequence of this is that anyone connected to these insecure devices can very easily manipulate their data and functions. This can range from mildly annoying, in the case of a thermostat being controlled remotely from a malicious source, to dangerous, in the case of pacemakers being instructed to stop. There have even been attacks on insulin pumps where malicious requests were sent to give the patient the maximum dose regardless of actual need [2]. There is a legitimate need to be able to easily build security into embedded devices, and protect consumer IoT devices from being compromised.

1.2 Project Objective

The objective of this project is to enable low power embedded applications and devices to easily and efficiently use high quality cryptography, by adding a chip to their board that "just works". The power consumption should be lower, and the speed of encryption faster, than if the cryptography were implemented purely in software. The encryption should have a simple C application program interface (API) that is hard to use incorrectly, which embedded device designers can integrate with their projects. The device should be simple to integrate into both new and existing embedded projects.

Sometimes these objectives are at odds with each other. When they are, the security of the design must not be compromised as a result. A cryptographic module that doesn't correctly provide security is worse than no cryptographic module at all, as it provides a false sense of security where there is none.

1.3 Block Diagram

The block diagram in Figure 1 details the components of the embedded security solution.

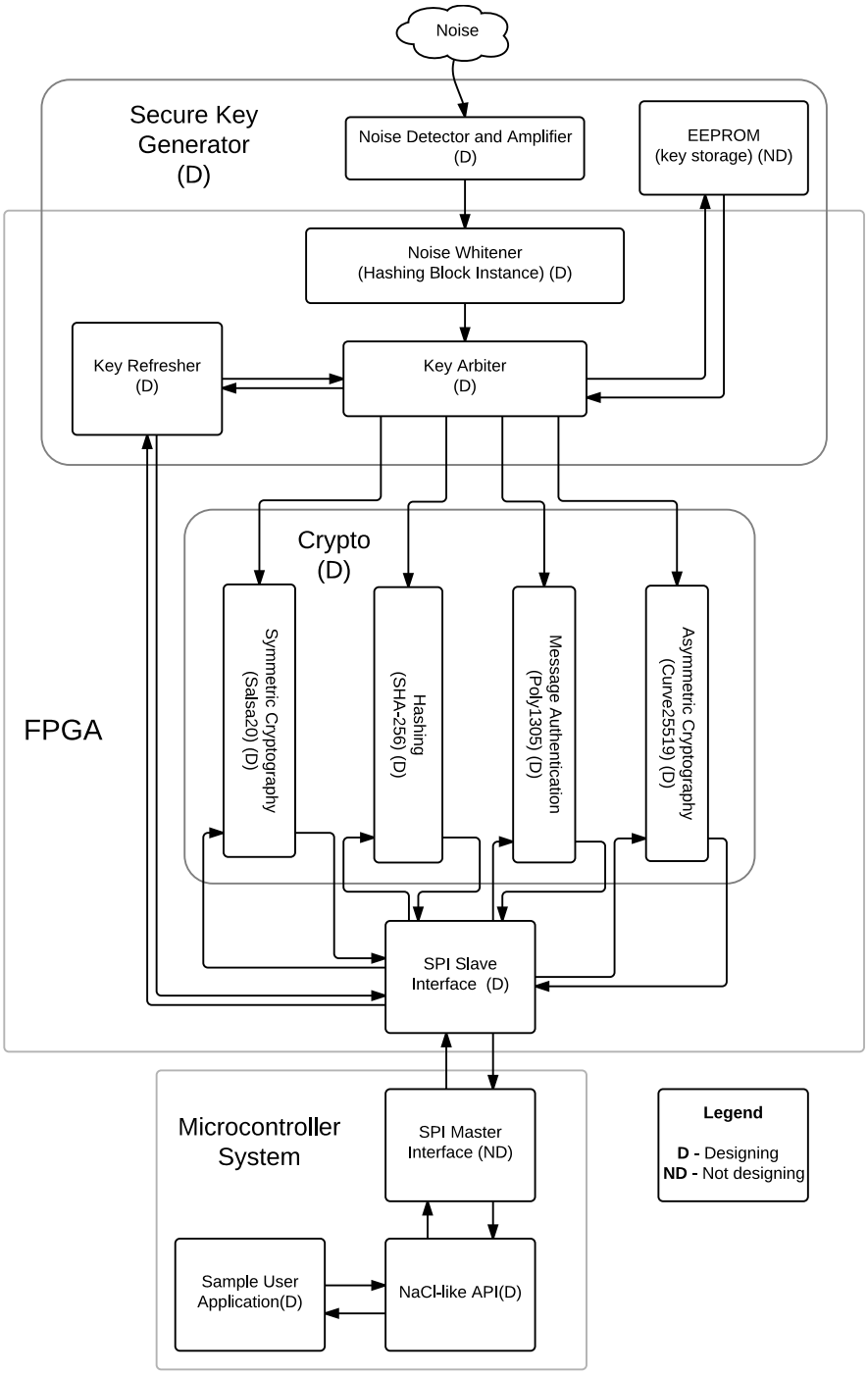

Figure 1: A block diagram showing the system's inputs, outputs, and abstract components.

1.3.1 Noise Detector and Amplifier

This module amplifies and digitally encodes thermal noise. A pseudo-random number generator is a function that creates numbers purely based off of an algorithm, and as such the numbers are predictable. By carefully tuning and filtering thermal noise, our random number generator (RNG) can be truly unpredictable.

1.3.2 Noise Whitener (Hashing Block Instance)

Noise may be random, but it may also be biased. Another instance of the hashing block will securely hash the random data, generating unbiased random data.

1.3.3 Key Arbiter

The key arbiter controls access to the private keys. Other subsystems request the keys from the key arbiter, which replies with a copy if they're authorized. This helps protect against key material leakage.

Note that the key material is never sent over the serial peripheral interface (SPI) bus, as those can be easily introspected with an oscilloscope.

1.3.4 Key Refresher

They Key Refresher provides an interface for the microcontroller to request a new private key, purging the old one. It responds to two different messages:

1. **Reset**, which generates a new keypair. The new public key is sent back.
2. **Pubkey**, which requests the chip's public key, such that other devices may send it secure messages.

The private key is never sent to the Key Refresher.

1.3.5 Hashing (SHA-256)

The Hashing block provides a field programmable gate array (FPGA) implementation of SHA-256 [3].

1.3.6 Message Authentication (Poly1305-Salsa20)

The Message Authentication block provides a hardware implementation of a variant of the Poly 1305-AES message authentication code (MAC) [4], but using Salsa20 instead of the advanced encryption standard (AES). It is a faster and more secure alternative to AES [5], and already implemented as the symmetric cryptography block.

1.3.7 Symmetric Cryptography (Salsa20)

The Symmetric Cryptography block provides a hardware implementation of Salsa20, a simple streaming, symmetric cipher [5].

1.3.8 Asymmetric Cryptography (Curve25519)

The Asymmetric Cryptography block provides a hardware implementation of the elliptic curve Curve25519 [6], used for Diffie-Hellman key exchange.

1.3.9 NaCL-like API

The networking and cryptography library (NaCL) provides a clear, hard to misuse C API to implement cryptographic functions in software [7]. The cryptographic hardware module will expose a C API aiming to replicate the NaCL API, providing a transparent transition for developers and applications from software to hardware cryptography.

1.3.10 SPI Slave Interface

All data exchange will be done via a SPI interface. The master will provide data to encrypt or decrypt. Control commands will also be sent via the SPI bus. A custom framing format will need to be specified to allow for the security module to know which operations to perform. It is most likely that the FPGA will not have dedicated SPI hardware onboard, meaning an interfacing component will have to be designed to work with the GPIO (general purpose input/output) pins on the board.

2 Project Specifications

Table 1 lists the functional specifications of our device: what it does. They are classified as essential and non-essential, where if an essential specification is not met the design is considered unsatisfactory.

Table 2 lists the non-functional specifications of our device: how it works. They are similarly classified as either essential or non-essential.

Table 1: Functional Specifications

Subsystem	**Specification**	**Additional Details**	**Necessity**
Secure Key Generator	This subsystem must be cryptographically secure.	In order to make sure the keys generated by the module are truly random and do not follow any sort of pattern, the keys generated will be subject to the Discrete Fourier Transform (Spectral) Test as outlined by the National Institute of Standards and Technology. A p-value is a measurement of how random a random number generator is. A value of 0 is completely non-random, and 1 is perfectly random. The p-value for the resulting from the test for the secure key generator must be greater than 0.1. [8]	Essential
Hashing Block (Sha256)	The hashing block must be able to calculate a Sha256 hash faster than a software implementation on an ARM Cortex M0 microprocessor.	There have been numerous studies done on the throughput of a Sha256 function on low power processors (specifically the Cortex-M0). The throughput of these studies maximizes at approximately 250kB/s. Our solution must be faster than this [9] [10].	Essential
Symmetric Cryptography Block (Salsa20)	The Salsa20 block must be able to encrypt and decrypt data faster than a software implementation of AES on an ARM Cortex M0 processor.	Salsa20 is a symmetric cipher that accomplishes goals similar to AES. AES implementations on Cortex-M0 have a throughput of 200 kB/s [11] [12].	Essential
Power Consumption	The power consumption of the prototype must be smaller than a Cortex-M0 for its software equivalents.	Unfortunately, no consistent studies currently exist to power profile the power usage of these cryptographic algorithms on ARM Cortex-M0 microcontrollers. As such, the group will have to perform this study to set an accurate benchmark for reasonable power performance. The benchmark should provide us with the power performance of SHA256, AES-128, Curve25519 and Poly-1305 on an ARM Cortex-M0 microcontroller. Our device must be beat the power performance found in this benchmark.	Non-Essential

Table 2: Non-Functional Specifications

Sub-system	**Specification**	**Additional Details**	**Necessity**
All	The device must be physically secure.	An attacker with physical access to the device should not be able to compromise the secret keys.	Essential
Secure Key Generator	It must be impossible for the central processing unit (CPU) to gain access to any secret cryptographic keys.	All cryptographic functions are performed on-device. Therefore, it should be possible to "air gap" the link between the CPU and the cryptography chip. This ensures that private keys are safe even if the CPU is compromised.	Essential
SPI Bus	The SPI bus clock line must be able to run at 100 kHz, 400 kHz, 1 MHz, 2 MHz and 5 MHz.	These varying speed settings are to ensure that almost any microcontroller capable of connecting to the Internet will be able to use the module.	Essential
Secure Key Generator	The system must generate a public/private key pair in less than 1000 ms.	If this is impossible without compromising security, it may take longer. This time is based off the time it takes for a Cortex-M0 to generate a key pair[13].	Non-Essential
Cryptographic Hardware Logic	The area of the FPGA used by the implementation of the cryptographic algorithms must be less than 22,000 logic elements.	This is the size of the Altera Cyclone IV, the device planned to be used. It is a device sized specifically for use in low-power, low-cost applications. The design should fit on this device.	Non-Essential
API	The API of the hardware must be identical to that of the Networking and Cryptography Library [7]	Having an identical API to that used in existing software solutions will make porting of existing designs trivial for users. NaCL has also been explicitly designed to be "hard to misuse", which is an extremely valuable property for a cryptography library.	Non-Essential

3 Risk Assessment

Table 3 contains a list of potential problems that our project may run in to, as well as its potential impact on our success and the likelihood of it happening.

Table 3: Risk assessment for the project

#	**Situations**	**Nature of Situations**	**Impact**	**Likelihood**
1	Project is too Complex to Complete on Time	This is related to time management, efficiency and skill levels of the team. Certain challenges may be discovered that are in fact active areas of research, and beyond the scope of what fourth year ECE students can accomplish within their senior year.To reduce the likelihood, we can work on our project during the semester preceding our design symposium. We can also manage our time efficiently to ensure we all contribute (on average) at least 10 hours of work each week.To reduce the harm, if we find ourselves short on time, we can partially implement the full NaCL API, and leave the rest in software.	High	Medium
2	Power Consumption is too Difficult to Accurately Estimate	This is related to possible insufficient skill or knowledge of members. We need to be able to accurately estimate (or measure if possible) the power usage to claim whether or not it is a low powered design, and thus usable by the devices we are making it for.To reduce the likelihood, we plan on both researching, and consulting with other professors who are familiar with FPGA development and prototyping to get a better idea of how to best achieve a good quantitative measurement.To reduce the harm, we plan on designing our prototype to be as efficient on power as we can while still maintaining functionality.	High	High
3	Power Consumption of Design is too High	This is also related to possible insufficient skill or knowledge of our members. The most important goals for this project is to prototype an application specific integrated circuit that uses less power than a software equivalent our design is not achieving one of its two primary purposes.To reduce the likelihood, (similar to the previous risk) we will be consulting with professors with expertise in this area and also peer-reviewed research papers on the topic.To reduce the harm we could remove the cryptography block that does asymmetric, as the symmetric cryptography block uses less power to encrypt.	High	Medium

References

[1] Hewlett Packard, "*Internet of Things Research Study*," 2014. [Online]. Available: http://h20195.www2.hp.com/V2/GetDocument.aspx?docname=4AA5-459ENW&cc=us&lc=en

[2] M. Zhang, A. Raghunathan, and N. Jha, "Trustworthiness of Medical Devices and Body Area Networks," *Proceedings of the IEEE*, vol. 102, no. 8, pp. 1174–1188, Aug. 2014.

[3] N. Sklavos and O. Koufopavlou, "Implementation of the sha-2 hash family standard using fpgas," *J. Supercomput.*, vol. 31, no. 3, pp. 227–248, 2005. [Online]. Available: http://dx.doi.org/10.1007/s11227-005-0086-5

[4] D. J. Bernstein, "The Poly1305-AES Message Authentication Code," *Fast Software Encryption*, vol. 12, no. 12, pp. 42–39, Mar. 2005. [Online]. Available: http://cr.yp.to/mac/poly1305-20050329.pdf

[5] D. J. Bernstein, "The Salsa20 Family of Stream Ciphers," *New stream cipher designs: the eSTREAM finalists*, Dec. 2007. [Online]. Available: http://cr.yp.to/snuffle/salsafamily-20071225.pdf

[6] D. J. Bernstein, "Curve25519: New diffie-hellman speed records." *Public key Cryptography*, no. 9, Apr. 2006, http://cr.yp.to/ecdh/curve25519-20060209.pdf.

[7] D. J. Bernstein, "Cryptography in nacl," *Journal of Cryptology*, Mar. 2009. [Online]. Available: http://cr.yp.to/highspeed/naclcrypto-20090310.pdf

[8] "A. Rukhin, J. Soto, et. al.", "A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications," NIST Standard, NIST, Special Publication 800-22, apr 2010. [Online]. Available: http://csrc.nist.gov/groups/ST/toolkit/rng/documents/SP800-22rev1a.pdf

[9] Cryptovia, *Hash Functions for ARM Thumb CPU*, http://www.cryptovia.com/ARM_Thumb_Hash.html, 2014.

[10] RealTimeLogic, *SharkSSL v2.3.3 benchmarks with ARM Cortex-M0@24MHz + ARM GCC 4.5.1*, 2015. [Online]. Available: https://realtimelogic.com/products/sharkssl/Cortex-M0/

[11] N. Mouha, B. Mennink, et. al., "Chaskey: An efficient mac algorithm for 32-bit microcontrollers," *Selected Areas in Cryptography*, 2014. [Online]. Available: https://eprint.iacr.org/2014/386.pdf

[12] D. J. Bernstein, *Why switch from AES to a new stream cipher?*, 2005. [Online]. Available: http://cr.yp.to/streamciphers/why.html

[13] L. Batina and M. Robshaw, *Cryptographic Hardware and Embedded Systems – CHES 2014: 16th International Workshop, Busan, South Korea, September 23-26, 2014, Proceedings*, ser. Lecture Notes in Computer Science / Security and Cryptology. Springer Berlin Heidelberg, 2014.